

The Urban Gazette

VOL. 1, NO. 024

2026-03-30

COVER STORY

The two kinds of error

In short: in my mind, errors are divided into two categories.

Evan Hahn

Expected errors (think “user entered invalid data”), which are part of normal operation, aren’t the developer’s fault, and should be handled. Unexpected errors (think “null pointer exception”) are the developer’s fault, likely indicate a bug, and are allowed to crash.

Error handling is an important, but often neglected, part of programming and user experience.

Over the years, I’ve developed an opinion about the two types of error in software. This is primarily informed by a career in web and application development, but I hope these learnings are widely applicable.

In my mind, errors are divided into two categories: expected and unexpected .

id=“expected-errors”>Expected errors

Expected errors happen during normal operation. Examples:

Validation errors when the user enters invalid data. You can’t control what the user types!

Network errors when the user’s network fails. It’s not your fault if the user turns their Internet off or has a slow connection!

Permission errors when your program isn’t allowed to do something. You can’t magically read forbidden files, fix a user’s password, or steal webcam privileges!

The developer hasn’t made a mistake when these happen, and there’s often little they can do to prevent it. The existence of these errors is not a bug (though failing to handle them can be). These aren’t the programmer’s fault.

Expected errors are recoverable. This might mean logging a warning, showing a message to the user, or using a fallback.

Expected errors should not throw , raise , or panic . Instead, they should return an error result. This works differently in every language, but is often a Result type, a union of null and the success value, or an error code. This pattern pushes you toward handling the error, which you should if you want to make your software reliable.

Expected errors should use WARN or INFO log messages because this isn’t a problem to solve. You may want to set up an alert if you start getting lots of warnings.

id=“unexpected-errors”>Unexpected errors

Unexpected errors should never happen. If they do, you’ve got a bug! Examples:

Assertion errors. For example, a function must be called with a non-empty string, and someone violated the contract if they didn’t.

Logic errors. If Thing A depends on Thing B, but Thing B isn’t properly initialized, that’s unexpected. Null pointer exceptions are also typically a surprise.

Invalid data errors. You can usually assume your database will give back valid data. If it doesn’t, you’ve probably got a bug somewhere.

You should generally not try to recover these errors. It’s okay to explode—crash, panic , and throw .

To get even more radical: I often think unexpected errors should completely crash the program . It’s disruptive in the short term, but I find crashes make software feel more reli-

able in the long run. You're more likely to hear about these problems from annoyed users—if not your own testing.

Unexpected errors should use `ERROR` or `FATAL` log messages because they indicate a real problem. At best, they indicate an incorrect assumption. At worst, there's a serious bug somewhere.

id="drawing-the-line">Drawing the line

The line between “expected” and “unexpected” depends on the task.

At one extreme: if you're making a prototype or quick script, I reckon all errors are unexpected. You might decide not to handle problems with the network, filesystem, or user input. Who cares? This is just a little script or idea.

At the other extreme: if you're coding for a space probe on a 50-year mission, almost all errors are expected, including catastrophic hardware failures .

Most programs lie somewhere in between, and you have to decide which errors are unexpected. For example, are memory allocation errors expected in your program? It depends.

In my experience, if you want to make your stuff more reliable, you'll trend toward expecting more and more errors. Lots can go wrong on a normal day! For example, my team recently had to deal with a memory allocation error, even though we're writing a Node.js app.

Some programming languages, like Rust and Zig, classify many errors as expected. Others, like JavaScript and Python, classify them as unexpected. For example, when you parse JSON in Go, the compiler makes you handle the error; not so in Ruby. I tend to prefer stricter compilers for production software and looser languages for scripts and prototypes, in part because of their philosophy about errors. (The Rustaceans among you probably notice that this whole post is very similar to Rust's error philosophy .)

To be clear: this is just what I think. I've found it useful to categorize errors this way. If you think about errors differently, (1) I'd love to hear it (2) I'm glad you're thinking about error handling in software.

See a response to this post, “The three kinds of error” .

{

FRONT PAGE

Corrupt a wish with GPT-4

@corrupt-a-bot GPT-4 bot · Sam Saffron

Granted! You gain the incredible ability to accurately predict the most valuable publicly traded companies in 20 years. However, as word of your gift spreads, everyone starts following your predictions and investing in those same companies. The sudden surge and overwhelming demand inflate their stock prices to absurd levels, creating massive financial bubbles.

As the bubbles continue to grow, the world economy destabilizes, and countries scramble to implement strict regulations on market activity. The introduction of these regulations inadvertently leads to even more economic turmoil, sparking a global recession that persists for decades. Your extraordinary gift ends up being the catalyst for worldwide financial devastation.

I wish I could teleport to any location in the world with just a blink of my eyes, but always arrive at the destination with a fabulous new outfit that suits the context and climate perfectly.

}

{

FEATURES

Debugging custom ApiGateway authorizers

Luciano Mammino · Luciano Mammino (loige)

Last week, while working on a custom REST API Gateway Lambda authorizer , I spent some time trying to debug a mysterious 500 error with a {"message":null} body. In this article, I will share why this was happening and how to debug this kind of error when building custom API Gateway authorizers.

id="the-use-case">The use case

Before diving into the problem, let me give you a bit of context about the use case I was working on. As I said, I was working on an open-source project implementing a custom authorizer for AWS API Gateway. The project is simply called oidc-authorizer and it's already available on GitHub and the Serverless Application Repository (SAR) . This authorizer is implemented as a Lambda function and it allows you to authenticate requests following to OIDC (OpenID Connect) protocol.

To better understand what that means, let's have a look at this lovely diagram (that I am proud to have drawn by myself):

In this diagram, we see a user that sends an authenticated request to API Gateway. API Gateway is configured to use a lambda as a custom authorizer. The lambda talks with a given OIDC provider to get the public key to validate the user token and responds to API Gateway to Allow or Deny the request.

This type of authentication is based on the idea that a token (in this case, a JWT) can be trusted only if it's signed by an authority we trust. That's why we need to talk with the OIDC provider to get the public key to validate the signature of the token.

I have been speaking and writing at length about JWT before, so I am just going to say that if you want to deep dive into the fascinating topic of OIDC you can check out the official specification of the OIDC standard . If you have the guts to read protocol specifications, it's a very interesting read, I promise! ☺

id="the-problem">The problem

Ok, all pretty cool, but what was the problem?

The problem is that I am a terrible programmer and when I started testing my first version of the authorizer my code was quite buggy and it didn't work in the way that REST API Gateway Authorizers are supposed to work.

I take responsibility for that, but that's only part of the problem. The other side of the coin is that AWS wasn't really giving me a useful error message: the only thing I could see when making a request was a 500 error with a response body containing only the JSON payload {"message":null} .

The full response:

Shrug... What do you do in these cases? ☹

You try to look for some logs in CloudWatch, right? Well, I did that and I found nothing. API Gateway doesn't have a log group by default and I was expecting to see some logs from my Lambda function, but there was nothing useful there!

id="the-solution">The solution

The solution is to enable extensive logging for API Gateway.

So, how do we do that?

We can do that using Infrastructure as code using CloudFormation or SAM.

These are the logical steps we need to follow:

Create a Role that allows API Gateway (at the service level) to write logs to CloudWatch

Use the AWS:: ApiGateway:: Account resource to assign the role we just created to API Gateway.

Updated the specific instance of a REST API Gateway stage to enable logging and tracing on it.

id="enabling-api-gateway-logging-to-cloudwatch-with-cloudformation">Enabling API Gateway logging to CloudWatch with CloudFormation

OK, let's start with the code for the first 2 steps using the following CloudFormation template:

To avoid overwriting other roles, we should only have one AWS:: ApiGateway:: Account resource per region per account, so we are creating a generic stack only to allow API Gateway to send logs to CloudWatch. We will be defining our APIs in other stacks.

Let's say we call this file apigw-logging.yaml . We can now deploy it with the following command:

Make sure to specify a region either with the -region flag or by setting the AWS_DEFAULT_REGION environment variable.

If all goes well, you should see an output like this:

Note : if you prefer to use SAM (which we will use later), you can use it to deploy the template above with the following command: sam deploy --guided --template apigw-logging.yaml .

id="enable-logging-and-tracing-for-a-specific-api-gateway-stage-using-sam">Enable logging and tracing for a specific API Gateway stage using SAM

Now that we have a role that allows API Gateway to write logs to CloudWatch, we can enable logging and tracing for

}

{

So long, and thanks for all the fish: how to escape the Linux networking stack

Chris Branch · Cloudflare Research

There is a theory which states that if ever anyone discovers exactly what the Linux networking stack does and why it does it, it will instantly disappear and be replaced by something even more bizarre and inexplicable.

There is another theory which states that Git was created to track how many times this has already happened.

Many products at Cloudflare aren't possible without pushing the limits of network hardware and software to deliver improved performance, increased efficiency, or novel capabilities such as soft-unicast, our method for sharing IP subnets across data centers. Happily, most people do not need to know the intricacies of how your operating system handles network and Internet access in general. Yes, even most people within Cloudflare.

But sometimes we try to push well beyond the design intentions of Linux's networking stack. This is a story about one of those attempts.

Hard solutions for soft problems

My previous blog post about the Linux networking stack teased a problem matching the ideal model of soft-unicast with the basic reality of IP packet forwarding rules. Soft-unicast is the name given to our method of sharing IP addresses between machines. You may learn about all the cool things we do with it, but as far as a single machine is concerned, it has dozens to hundreds of combinations of IP address and source-port range, any of which may be chosen for use by outgoing connections.

The SNAT target in iptables supports a source-port range option to restrict the ports selected during NAT. In theory, we could continue to use iptables for this purpose, and to support multiple IP/port combinations we could use separate packet marks or multiple TUN devices. In actual deployment we would have to overcome challenges such as managing large numbers of iptables rules and possibly network devices, interference with other uses of packet marks, and deployment and reallocation of existing IP ranges.

Rather than increase the workload on our firewall, we wrote a single-purpose service dedicated to egressing IP packets on soft-unicast address space. For reasons lost in the mists of time, we named it SLATFATF, or "fish" for short. This service's sole responsibility is to proxy IP packets using soft-unicast address space and manage the lease of those addresses.

WARP is not the only user of soft-unicast IP space in our network. Many Cloudflare products and services make use of the soft-unicast capability, and many of them use it in scenarios where we create a TCP socket in order to proxy or carry HTTP connections and other TCP-based protocols. Fish therefore needs to lease addresses that are not used by

open sockets, and ensure that sockets cannot be opened to addresses leased by fish.

Our first attempt was to use distinct per-client addresses in fish and continue to let Netfilter/conntrack apply SNAT rules. However, we discovered an unfortunate interaction between Linux's socket subsystem and the Netfilter conntrack module that reveals itself starkly when you use packet rewriting.

Collision avoidance

Suppose we have a soft-unicast address slice, 198.51.100.10:9000-9009. Then, suppose we have two separate processes that want to bind a TCP socket at 198.51.100.10:9000 and connect it to 203.0.113.1:443. The first process can do this successfully, but the second process will receive an error when it attempts to connect, because there is already a socket matching the requested 5-tuple.

Instead of creating sockets, what happens when we emit packets on a TUN device with the same destination IP but a unique source IP, and use source NAT to rewrite those packets to an address in this range?

If we add an nftables "snat" rule that rewrites the source address to 198.51.100.10:9000-9009, Netfilter will create an entry in the conntrack table for each new connection seen on fishtun, mapping the new source address to the original one. If we try to forward more connections on that TUN device to the same destination IP, new source ports will be selected in the requested range, until all ten available ports have been allocated; once this happens, new connections will be dropped until an existing connection expires, freeing an entry in the conntrack table.

Unlike when binding a socket, Netfilter will simply pick the first free space in the conntrack table. However, if you use up all the possible entries in the table you will get an EPERM error when writing an IP packet. Either way, whether you bind kernel sockets or you rewrite packets with conntrack, errors will indicate when there isn't a free entry matching your requirements.

Now suppose that you combine the two approaches: a first process emits an IP packet on the TUN device that is rewritten to a packet on our soft-unicast port range. Then, a second process binds and connects a TCP socket with the same addresses as that IP packet:

The first problem is that there is no way for the second process to know that there is an active connection from 198.51.100.10:9000 to 203.0.113.1:443, at the time the connect() call is made. The second problem is that the connection is successful from the point of view of that second process.

It should not be possible for two connections to share the same 5-tuple. Indeed, they don't. Instead, the source address of the TCP socket is silently rewritten to the next free port.

}

{

JavaScript, low-level or AI?

Luciano Mammino · Luciano Mammino (loige)

The software industry is going to be fun to see in the coming 5-10 years.

I see an interesting tension happening right now...

id="the-generative-ai-side">The generative AI side

On one side we have generative AI capturing almost every bit of the software development lifecycle and effectively becoming a new high-level abstraction, possibly the highest level we have ever seen in our industry.

There seems to be no stopping this phenomenon... just looking at the latest announcements from GitHub Universe, it's clear that we will have to adopt AI in one way or another and this is not necessarily a bad thing if it truly makes us all more productive and focused on generating business value.

If you don't know what I am talking about you should check out this video:

style="padding-bottom: 56.25%; height: 0; margin-bottom: 5em;">

And if that's not enough you should watch the full GitHub Universe 2023 keynote .

GitHub Universe 2023, Day 1. Thomas Dohmke on stage with a slide in the background saying "One more thing", mimicking the Steve Job's launch of the iPad

The part that impressed me the most is that now Copilot can also help you lay out the structure of a project, breaking down requirements into potential tasks. Once you are happy with the result, it can start to work on the individual tasks and submit PRs. This is called Copilot workspace , if you want to have a look.

It seems too good to be true and it's probably going to be far from perfect for a while, but there's great potential for efficiency here, and I am sure GitHub (and other competitors) will keep investing in this kind of products and maybe in a few years, we'll be mostly reviewing and merge AI-generated PRs for the most common use cases.

If you think that ChatGPT was launched slightly less than 1 year ago, what will we be seeing in 5 or 10 years from now?

id="the-low-level-side">The low-level side

On the other side, we have a wave of new low-level languages such as Go , Rust , Zig (and Carbon , and Nim , and Odin , and VLang , and Pony , and Hare , and Crystal , and Julia , and Mojo , and.. I could keep going here... ☺).

OK, I really wanted to put the lovely Hare language mascot here. Hare is a systems programming language designed to be simple, stable, and robust. Hare uses a static type system,

manual memory management, and a minimal runtime. It is well-suited to writing operating systems, system tools, compilers, networking software, and other low-level, high-performance tasks.

All these languages take slightly different trade-offs, but, at the end of the day, they are built on the premise that we need to go lower level and have more fine-grained control over how we use memory, CPU, GPU and all the other resources available on the hardware. This is perceived as an important step to achieve better performance, lower production costs, and reach the dream of "greener" computing.

If you are curious to know why we should be caring about green computing , let's just have a quick look at this report: Data Centres Metered Electricity Consumption 2022 (Republic of Ireland) .

The report findings state that in 2022, in Ireland alone, data centers' energy consumption increased by 31%. This increase amounts to an additional 4,016 Gigawatt/hours. To put that in perspective we are talking about the equivalent of an additional 401.600.000.000 (402 billion!) LED light bulbs being lit every single hour . If you divide that by the population of the Republic of Ireland this is like every individual is powering 80.000 additional LED light bulbs in their home, all day and night! And this is just the increase from 2021 to 2022... How friggin' crazy is that?! ☹

Ok, now one could argue that we had low-level languages pretty much since the software industry was invented. So why isn't that the default and why do we bother wasting energy with higher-level programming languages?

That's actually quite simple: because coding in low-level programming languages such as C and C++ is hard! Like really really hard! And it's also time-consuming and therefore expensive for companies! And I am not even going to mention the risk of security issues that come with these languages.

So why should this new wave of low-level programming language change things?

It's a bet that we can achieve better performance and efficiency by going lower level and deciding to be explicit about the minutia of how we want to use the hardware at best.

Luciano Mammino

Well, my answer is that they are trying to make low-level programming more accessible and safe. They are trying to create paradigms that could be friendly enough to be used for general computing problems (not just low-level), which could potentially bring the benefits of performance and efficiency even in areas where historically we have been using higher-level languages and made the hard tradeoff of fast development times vs sub-optimal performance.

}

{

Invite-only microsites with Next.js and AirTable

Luciano Mammino · Luciano Mammino (loige)

I recently needed to create an invite-only Next.js-powered microsite (for my wedding 🎉) and in this article I'm going to show you how I went about creating invite codes and how I implemented code validation in the app, using Next.js API endpoints and AirTable as a lightweight backend.

id="the-use-case-and-the-tech-stack">The use case and the tech stack

24 June 2022 was the best day of my life.

Seriously, I got married with the human being I love the most and everything was just perfect.

But we are not here to talk about my wedding, right?

Well, we kinda are, but in a nerdy kind of way that only software engineers can (hopefully) appreciate. OK, don't tell my wife I said that... 🤫

Specifically, we will discuss how to implement an invite-only microsite to share information with all the people invited to a private event.

Just to put things in context, let's wear the PM hat for a second, and let's discuss what kind of requirements did I impose on myself while building my wedding website:

I wanted to be able to iterate quickly, mostly focusing on content and design, not much on infrastructure.

I wanted to build a simple solution that would be easy to host, maintain and update.

I wanted to have a very lightweight backend, possibly something managed.

Even better if I could give access to the data storage to someone else (my wife) so that we could collaborate on managing the data about the guests.

If I could host everything very cheaply (if not even freely), that would be great.

For all the reasons above I settled for adopting the following pieces of tech:

Next.js for building a statically rendered frontend and the backend APIs

A private GitHub repository for managing the code of the Next.js app

Vercel for hosting the project

AirTable to store and manage all the data (invite codes, guests, RSVP, etc.)

Keep in mind that Next.js is free and open source, GitHub can be used for free (even if you want to keep your repo private), and Vercel and AirTable have both generous free plans that are more than enough for this kind of use case.

With the stack above I ended up spending 0 money and I was able to complete the project in a couple of weekends! 🎉

id="making-a-nextjs-site-private">Making a Next.js site private

On top of the requirements discussed above, my wedding website had 2 additional constraints:

Every different guest should see something different (the website would look like a custom invite letter for them).

People without an invite code should not be able to access any content on the website.

The following illustration shows how I went about making that happen in the context of a React SPA (Single Page Application):

Please boost my ego for a second, isn't this a beautiful illustration? 😊

OK, let's be serious:

The user loads the application by visiting the website with a personal invite code (passed as a query string parameter). When the page is loaded the React application starts on the client side. The app displays only a loading spinner at this stage.

The first thing that the application does is to read the invite code from the URL. At this point, it can call the backend (a Next.js API endpoint running on Vercel) to get the invite code validated. In turn, the backend requests the data storage (an AirTable spreadsheet) to check if the code exists (in which case it's valid). If the code exists, the backend also returns the custom information associated with that invite code. For instance, the name of the invited person.

If the code is valid, the application has all the information to render the private website.

If it is not valid, an error message is displayed instead.

id="lets-build-an-invite-only-website">Let's build an invite-only website!

No, I am not going to show you my private wedding website (sorry, not sorry)!

Instead, we are going to build a more fictional example in this article: an invite-only website for a secret pizza party. Here's what different invites will look like:

As you can see from the picture, every guest (with a valid invite code) will get a custom invite page with their name,

}

{

Integrating Twig.js and BazingaJsTranslationBundle

Luciano Mammino · Luciano Mammino (loige)

Recently I had the need to run a twig template that uses the trans filter on my frontend using twig.js , a pure JavaScript port of twig written by the good Johannes Schmitt . The JavaScript version does not handle all the functionalities offered by the original PHP version (even if it goes pretty close) and in particular it does not natively handle the trans filter.

So, at first, I got a JavaScript runtime exception on my page when trying to use the template. Luckily enough the JavaScript version of twig is extensible like the PHP one and it is very easy to add new filters and functions.

In my specific case I had a Symfony application where I was already using BazingaJsTranslationBundle to manage dynamic translations on the frontend. As I discovered the twig.js extensibility, it was very easy to start building a twig.js extension by using the Translator JavaScript object offered by the Bazinga bundle.

Note : I will not go into the details about how to install twig.js and BazingaJsTranslationBundle in a Symfony application. You can find all the needed informations on their websites/github pages.

In my first attempt I wrote something like this:

That seemed to work pretty good until I started to use translation strings with parameters. Parameters were not replaced with their respective values! The problem laid in a subtle difference on how the BazingaJsTranslationBundle and the standard twig handle parameters. Let's see a simple example.

Suppose we have the string `hello %name%` . With twig we expect to do something like:

Note the % delimiters around the parameter name.

The Translator.trans method expects an hash map without parameter delimiters in it. So we would have to do something like this:

Note that there's no % delimiter this time. The Translator.trans method manages the detection of parameters by itself and you can also decide to customize the delimiters by setting the values: `Translator.placeholderPrefix` and `Translator.placeholderSuffix` . Obviously I suggest you to be consistent and use the same placeholders you use with PHP (especially if you need to share templates and translations from the backend to the frontend).

So my final solution was the following:

This way it automatically normalizes parameters for the Translator object (by removing any delimiter from parameter keys) and I have a consistent behavior between twig and twig.js. My normalization approach is very rough and you

can surely find a better approach (maybe using a regex). Let me know if you do it ;)

Obviously you can also avoid the normalization and keep the responsibility to pass the parameters hash map in the way the Translator object expects it (without delimiters). In this case you can stick to my first implementation.

That's all. See ya ;)

}

{

Why Legislators Need Technologists

Steve Bellovin

A rather bizarre bill has been introduced in the Michigan legislature, the Anticorruption of Public Morals Act, H. B. 4938 . While there's a lot to object to in the bill, I'll leave the broader criticisms to others and focus on some technology issues.

The goal of the bill is specified in §3(1): "A commercial entity, public institution, private actor, or internet platform shall not knowingly distribute or make available prohibited material"—basically, their perception of pornography—"via the internet to any individual in this state." Even legislators know that location-spoofing is easy, so §3(5) bars the sale of "circumvention tools"—and that's where the trouble starts.

§2(a) defines "circumvention tools" as "any software, hardware, or

service designed to bypass internet filtering mechanisms or content

restrictions including virtual private networks, proxy servers, and

encrypted tunneling methods to evade content restrictions."

"Designed to bypass" and "to evade" are doing a lot of work here, but

the technologies named are most certainly vital and multi-use.

Virtual private networks (VPNs), for example, go back at least to

SP3 , a

US government design intended to protect communications on the Internet.

I wrote about a mechanism to create VPNs in

1990 .

Most important, the IETF defined some VPN protocols in IPsec

(1998).

None of these were designed to "bypass" filtering or "evade" content restrictions, because there were no such things back then.

Rather, they were designed to provide broad traffic protection and

to extend corporate networks beyond the firewall.

And tunneling? It's been part of ssh since its beginning, in 1996. Again, there was no conception of evading content restrictions.

VPNs and ssh tunnels are vital business tools—but this bill requires Michigan ISPs to "actively monitor and block known circumvention tools" (§3(3)). Not only ordinary businesses use them, ISPs use them to manage their infrastructure. This bill might outlaw secure operation of any ISP in the state, to say nothing of business travelers to Michigan.

Can this detection even be done? Well, if you use standard port and protocol numbers, you can detect ssh and IPsec, but there's no requirement to do either. This implies using deep packet inspection on all traffic, which is hideously expensive and trivial to bypass.

It gets worse. §4, which applies to any "internet platform, website, or social media service that is accessible by a user in this state", imposes a pile of restrictions. "Any website... accessible by a user in this state" is basically the entire Internet—but every such site has to comply. The Michigan-specific filtering has to be applied "uniformly across all users," presumably including those not in Michigan. Everyone has to implement content moderation tools, except that automated ones don't work and human ones don't scale. And of course, every web site on the planet has to file an annual report with the Michigan state police. I wonder how many languages the state police can read—the bill doesn't seem to require that the reports be in English...

In a minor vein, §2(f)(ii)(A) exempts "peer-reviewed academic content". I suspect that the people behind this bill have never heard of, e.g., arxiv.org , a preprint site. Most (but not all!) content there is intended for eventual peer review, but it hasn't been peer-reviewed yet. Subsection (B) exempts "material to be used for scientific and medical research or instruction", but not all material there is intended to further future scientific research.

In short: even if getting "pornography" were a good idea, this bill is a horrible way to go about it.

}

{

Introducing flickr-set-get a command line app to download photos

Luciano Mammino · Luciano Mammino (loige)

I recently developed a small command line app that allows you to download an entire gallery from Flickr, it's called flickr-set-get and you can find it on NPM and GitHub .

id="why">Why?

To be honest I had myself the need to download a large set of photos (more than 400 photos) from Flickr and I didn't wanted to do it manually. I also wasn't able, after a quick search, to find something simple to solve this task. Given that I am currently getting into deep of Node.js this was the perfect chance to develop something practical.

id="how-its-built">How it's built?

As I said it's built using Node.js. As part of my learning process, I tried to not reinvent the wheel and to follow "the Node way" so I used some very good modules as the foundation to build this app:

Commander , Cli and Prompt for the command line facilities (Options/Arguments parsing, help generation, progress bar, receiving input from the user)

Request : to simplify the creation of the Http request to call the Flickr APIs

Async : to manage all the asynchronous tasks in parallel and with a configurable level of concurrency (one of the features that in my humble opinion makes node/javascript stand out from the crowd of the interpreted programming languages).

As development tools I also used:

JSCS : to check the code standard I adopted

Mocha and Chai : test runner and assertion frameworks for the unit testing

Nock : amazing module to "mock" the Http requests and avoid to hit Flickr servers in my tests

Istanbul : for the code coverage

Travis CI : continous integration service

Coveralls : service to keep track of the coverage changes after every new commit

Gemnasium : service that checks your dependencies and alerts you if they get out of date

id="going-asynchronous-and-execute-requests-in-parallel">Going asynchronous and execute requests in parallel

First of all you need to call the flickr.photosets.getPhotos API method to find out the IDs of all the photos in the set.

Then, to find out the URL of every photo you need to call the flickr.photos.getSizes API method. This method gives you all the URLs to download all the different sizes of a given photo. Once you got the URL you can simply use that to download a photo into your local drive.

The first API call is the starting point and then, since you now have all the IDs, you can make all the other requests in parallel.

Thanks to Async it was very easy to create a task function that figured out the specific URL of a given photo and then downloads it into a file. Then I just needed to replicate this task for every photo and run all of them in parallel. Thanks to the Async.parallelLimit() function it's even possible to run the tasks with a configurable concurrency level.

You can find out how I implemented this specific logic on GitHub: <https://github.com/lmammino/flickr-set-get/blob/ecaf0d8bea5791cf24c4b71791a7eb1fd2e60bcb/lib/Flickr.js#L212-L253> .

id="how-to-use-it">How to use it?

If you're interested on using it I suggest you to read the official documentation on GitHub , but it should be quite easy to understand and use it if you are a developer (and it's not meant to be used by other people who are comfortable with the command line!)

id="current-status">Current status

The project has been developed in a weekend as a "learning project" and tested just by me on few Flickr galleries. It's to be considered ALPHA quality and I expect it to be fully of bugs and uncovered edge cases. It's a small side project anyway so I don't expect to put so much effort on it again unless it gets adopted by a relevant number of people (which, quite frankly, is really unlikely to happen). Being an open source project I obviously tried to put in place all the basic infrastructure (tests, code standards, repository, versioning, etc.) to make it easy for other people to work on improving it. Again, more precise informations are included in the official documentation .

id="opinions">Opinions

For me it has been a really good learning experience, especially considering the fact that it was a good scenario to use asynchronous code dealing with several parallel Http requests. As I still consider it a learning experience I would really love you (probably more experienced than me with Node.js) to give some opinions, critics, new ideas. I offer the comment box below for this sake... please don't be too much indulgent : D

}

{

2018 - A year in review

Luciano Mammino · Luciano Mammino (loige)

It is that time of the year when I have to look back at the previous year and see what were my achievements, my failures and set my expectations for my 2019.

Full disclosure : I write this type of posts mostly for myself, so I expect this to be boring to death and most likely useless for everyone else.

You have been warned, read at your own risk!

id="static-blog-migration">Static blog migration

Interesting achievement of the last year is that I finally migrated this blog from Ghost to a static setup using totally free and Open Source tools, which costs me 0.00 \$ per month .

I am using GatsbyJS as the engine for the static website.

If you are curious to see my setup (and maybe even correct typos in my articles), I have everything open sourced at [lmammino/loige.co](https://github.com/lmammino/loige) .

I don't think my GatsbyJS config is particularly interesting, but the entire publishing pipeline might be worth some words.

As per January 2019, this blog is published on GitHub pages using the gh-pages branch which I publish using the very handy gh-pages package .

The static website is served through Cloudflare CDN , which takes care of speeding up the delivery across the globe and other handy things like making sure the website runs on https correctly.

Every time I push on master, a CircleCI build is triggered, and, if the build succeeds, a new version of the website is published automatically.

Checkout my CircleCI config if you want to do something similar.

I wanted to do something like this for a while, so being able to achieve this was a big win for me!

id="conference-talks">Conference talks

Pretty much like in the last 2 years, I tried to invest some of my time in engaging with the community through conference talks.

This year wasn't my best, but I still managed to deliver 8 talks and a workshop around the world. For the first time I delivered one outside Europe in the amazing New York.

Here's the full list of talks for 2018, you can find more details in the usual speaking section with links to slides and videos when available:

"Getting started with Serverless and Lambda Functions" (workshop), Codemotion Rome (April)

"Unbundling the JavaScript module bundler", Codemotion Rome (April)

"The future will be Serverless", JS Day Verona (May)

"Cracking JWT tokens: a tale of magic, Node. JS and parallel computing", Web Rebels Oslo (June)

"Cracking JWT tokens: a tale of magic, Node. JS and parallel computing", Buzz JS New York (June)

"Unbundling the JavaScript module bundler", Dublin JS (July)

"Cracking JWT tokens: a tale of magic, Node. JS and parallel computing", FullStack London (July)

"Unbundling the JavaScript module bundler", Øredev Malmö (November)

"Processing TeraBytes of data every day and sleeping at night", Codemotion Milan (November)

Compared to 2017, when I presented 17 talks/workshops, this is a -52.94% "growth" rate. Ouch! ☹

In reality, I don't mind at all this level of commitment for conferences and workshops and I'll try to keep 2019 to a similar rate to 2018 and not overcommit.

id="career">Career

From a career perspective I spent my 2018 at Vectra, where I worked "automating the hunt for cyberattackers and speeding-up incident response", which essentially means building software that help companies to identify cyber attacks and resolve them as quick as possible and with the least amount of effort.

One year at Vectra has been a blast and I had the pleasure (and luck ☺) to be able to contribute to some important success stories:

I was involved in bootstrapping and scaling up a new European site that started from 3 and now counts 20+ people (engineering and sales). All world-class professionals with whom I love to work and from whom I can learn a lot every day.

Built a new product line from scratch. The product is already making revenue! This is a quite complicated product that is able to ingest and make searchable Terabytes of data every day !

Witnessed an astounding 104% growth in annual recurring revenue .

One year in the making, there was no shortage of personal skills development:

}

{

How to build your own VPN, or: the history of WARP

Chris Branch · Cloudflare Research

Linux's networking capabilities are a crucial part of how Cloudflare serves billions of requests in the face of DDoS attacks. The tools it provides us are invaluable and useful, and a constant stream of contributions from developers worldwide ensures it continually gets more capable and performant.

When we developed WARP, our mobile-first performance and security app, we faced a new challenge: how to securely and efficiently egress arbitrary user packets for millions of mobile clients from our edge machines. This post explores our first solution, which was essentially building our own high-performance VPN with the Linux networking stack. We needed to integrate it into our existing network; not just directly linking it into our CDN service, but providing a way to securely egress arbitrary user packets from Cloudflare machines. The lessons we learned here helped us develop new products and capabilities and discover more strange things besides. But first, how did we get started?

A bridge between two worlds

WARP's initial implementation resembled a virtual private network (VPN) that allows Internet access through it. Specifically, a Layer 3 VPN – a tunnel for IP packets.

IP packets are the building blocks of the Internet. When you send data over the Internet, it is split into small chunks and sent separately in packets, each one labeled with a destination address (who the packet goes to) and a source address (who to send a reply to). If you are connected to the Internet, you have an IP address.

You may not have a unique IP address, though. This is certainly true for IPv4 which, despite our and many others' long-standing efforts to move everyone to IPv6, is still in widespread use. IPv4 has only 4 billion possible addresses and they have all been assigned – you're gonna have to share.

When you use WiFi at home, work or the coffee shop, you're connected to a local network. Your device is assigned a local IP address to talk to the access point and any other devices in your network. However, that address has no meaning outside of the local network. You can't use that address in IP packets sent over the Internet, because every local IPv4 network uses the same few sets of addresses.

So how does Internet access work? Local IPv4 networks generally employ a router, a device to perform network-address translation (NAT). NAT is used to convert the private IPv4 network addresses allocated to devices on the local-area network to a small set of publicly-routable addresses given by your Internet service provider. The router keeps track of the conversions it applies between the two networks in a translation table. When a packet is received on either network, the router consults the translation table

and applies the appropriate conversion before sending the packet to the opposite network.

Diagram of a router using NAT to bridge connections from devices on a private network to the public Internet

A VPN that provides Internet access is no different in this respect to a LAN – the only unusual aspect is that the user of the VPN communicates with the VPN server over the public Internet. The model is simple: private network IP packets are tunnelled, or encapsulated, in public IP packets addressed to the VPN server.

Schematic of HTTPS packets being encapsulated between a VPN client and server

Most times, VPN software only handles the encapsulation and decapsulation of packets, and gives you a virtual network device to send and receive packets on the VPN. This gives you the freedom to configure the VPN however you like. For WARP, we need our servers to act as a router between the VPN client and the Internet.

NAT's how you do it

Linux – the operating system powering our servers – can be configured to perform routing with NAT in its Netfilter subsystem. Netfilter is frequently configured through `nftables` or `iptables` rules. Configuring a “source NAT” to rewrite the source IP of outgoing packets is achieved with a single rule:

```
nft add rule ip nat postrouting oifname “eth0” ip saddr 10.0.0.0/8 snat to 198.51.100.42
```

This rule configures Netfilter's NAT feature to perform source address translation for any packet matching the following criteria:

The source address is the 10.0.0.0/8 private network subnet - in this example, let's say VPN clients have addresses from this subnet.

The packet shall be sent on the “eth0” interface - in this example, it's the server's only physical network interface, and thus the route to the public Internet.

Where these two conditions are true, we apply the “snat” action to rewrite the source IP packet, from whichever address the VPN client is using, to our example server's public IP address 198.51.100.42. We keep track of the original and rewritten addresses in the rewrite table.

Schematic of an encapsulated packet being decapsulated and rewritten by a VPN server

You may require additional configuration depending on how your distribution ships `nftables` – `nftables` is more flexible than the deprecated `iptables`, but has fewer “implicit” tables ready to use.

You also might need to enable IP forwarding in general, as by default you don't want a machine connected to two dif-

}

{

Migrating from Gatsby to Astro

Luciano Mammino · Luciano Mammino (loige)

I recently migrated this blog from Gatsby to Astro . Finally, I have to say! I have been meaning to move away from Gatsby for a while, but I never really prioritised this activity until now and all my previous attempts had failed for one reason or another...

When I published this new version, quite a few people reached out to ask various questions related to this migration. So, in this article I will try to cover my motivations for migrating, the process I followed and some issues and unexpected things I learned along the way. The migration also included a complete redesign of the blog, so I will also touch on that aspect a little bit.

id="the-motivations">The motivation(s)

I have been running this blog for 10 years now. I initially started with a simple WordPress deployment, which I didn't tolerate for very long. So I quickly moved to Ghost , which I self-hosted on a Digital Ocean VM and ran for a few years. I was quite happy with it, except I wanted a more serverless low-maintenance solution, so I eventually managed to move this blog to Gatsby and live happily with a fully static and effectively costless solution. This was in 2018, so I have been a Gatsby user for about 6 years!

So what's wrong with Gatsby and what was the motivation to move away from it?

Actually, Gatsby in itself is quite a good product. It has tons of features and plugins. If you like writing React and MDX and shipping your site as a SPA (Single Page Application) nicely cached with a built-in PWA (Progressive Web Apps), Gatsby has it all! It's also very easy to get started with and it has a very active community publishing a plethora of plugins for the most disparate use cases.

OK, Luciano, so why the heck did you want to move away from it, then?!

Good question, I am glad you asked! I think it's hard to pinpoint one reason; it's most likely a combination of a few things which are not always Gatsby's fault...

id="keeping-up-with-the-upgrades">Keeping up with the upgrades

The first releases of Gatsby moved quite quickly and they often required a significant amount of work in migrating to the next version. At some point, I stopped caring about upgrades and I eventually got stuck on an unsupported version. This eventually required me to build my blog with an old and unsupported version of Node.js and to have to rely on outdated and insecure dependencies. Sure, it's a static blog, so it's not like I am publishing an interactive application that might have tons of vulnerabilities, but, still, it's annoying to rely on an outdated build system and have to switch Node version every time I need to work on the blog.

Fun fact: by looking at the number of weekly Gatsby downloads by version , it looks like I was not alone with this problem!

As you can see, at the time of writing this, Version 5 (the current major release) is way behind versions 2, 4, and 3 respectively in terms of weekly downloads... Funny to see that the majority of users are still running on version 2! I was stuck on version 3, so maybe I wasn't doing too bad after all! ☹

id="complexity">Complexity

This is probably a matter of personal preference, but I feel that Gatsby is a bit of a complex beast. If you need a good reason to justify this statement I actually have 2: GraphQL and Client-side hydration . Let me explain...

Let's start with GraphQL.

When you build static websites, you generally need to have some kind of way to load data from one or more sources. After all, the main task of a static site generator is to take data and templates as input and produce web assets (HTML, CSS, JavaScript, Images, etc) as output.

Gatsby abstracts the concept of data loading with GraphQL. This is a very powerful abstraction that allows you to fetch data with extreme precision. Many frontend developers have gotten accustomed to this way of retrieving data so it makes sense for Gatsby to adopt this abstraction.

On the other hand, it feels like an unnecessary complexity when dealing with static websites where most often you are loading data from relatively simple markdown, MDX or Yaml files. Most often you just want to load all the data (you'll need to generate all the pages anyway) and you still have to think in terms of GraphQL queries.

I did enjoy it at first (maybe because of the novelty of GraphQL), but I got eventually fed up with it!

What about Client-side hydration? Isn't that a good thing?!

Giving you an overview of the Gatsby build process is a bit out of the scope of this article, so I am going to focus only on what the final output looks like.

Gatsby produces highly optimised static websites that are built as a Single Page React Application with Server Side rendering and Client Side hydration. This means that the initial page load is very fast and the subsequent navigation is also very fast because the pages are already rendered and only need to be hydrated with the data.

This is all very good and dandy! Many would argue that this is the state of the art of frontend development and I don't necessarily disagree with that. However, it's not a simple system and it comes with lots of complexity and sometimes can lead to very nasty and unexpected bugs that are very hard to understand and fix.

}

{

2021 - A year in review

Luciano Mammino · Luciano Mammino (loige)

A post where I reflect on my professional life in 2021 as a look ahead for what to expect in 2022.

As usual, let me tell you that I write this kind of posts for myself and I don't expect other people to be able to read this and be engaged. So, if you really feel like you want to venture forward and have a peek on my 2021, well, I am flattered... but make sure to grab a coffee, because this might get a little (just a little...) boring!

Yes, this GIF is probably the least boring element of the whole article! ☺

id="joining-fourtheorem">Joining fourTheorem

The biggest highlight of my 2021 is that in February I joined fourTheorem as a Senior Architect .

If you never heard about fourTheorem, here's what you need to know. It's a great service company founded by Peter Elger , Fiona McKenna and Eoin Shanaghy .

The company, while still relatively new and small, has a large amount of expertise in serverless , platform modernisation and AI as a service . Most importantly, fourTheorem has a huge amount of experience with AWS . We are AWS Consulting Partners and we have been helping small and big customers to transition to the cloud and get the best out of it!

I have to admit I was initially skeptical about being able to work with a company that is not focused on delivering one single product but that is mainly helping many other companies to do that. In the past I rarely had a great experience when being on the other side and dealing with other service companies.

But I was immediately reassured by the fact that Eoin and Peter are not just two amazingly good technical founders, but they also have an impressive track record working with companies of all sizes: from small startups to big corporations. Together with Fiona, the founding trio has a very inspiring vision for what they want the company to become.

I immediately had the feeling that they wanted to build something very different from the other service companies out there. This feeling became apparent pretty quickly just after the first week, when I saw how they managed to embed their team into the customer's team and work together in all the phases of the product. FourTheorem can literally be an extension of a team, bringing not just people and technology skills, but also a very practical and agile approach to software delivery.

And after almost a year, I cannot hide the fact that I have never been happier in a technical position. I can work in an environment with fantastic colleagues, an amazing culture, in a very supportive environment where I am learning a ton.

Most importantly, I feel like there is a lot of potential and that I can have an impact.

Finally I get a significant amount of "work time" to invest in content creation, open source and to study (new tech, certifications, etc.). This is something that I consider integral part of my professional career path and it's great to see that it is valued and supported!

Mandatory disclaimer: we are always hiring, so if you are interested feel free to reach out to me on Twitter or LinkedIn .

Also... if you are looking for help for your cloud migration or to improve your AWS deployments, don't be shy, let's have a coffee-chat ! ☺

Be not afraid of discomfort. If you can't put yourself in a situation where you are uncomfortable, then you will never grow. You will never change. You'll never learn.

— Jason Reynolds

id="becoming-an-mvp">Becoming an MVP

In 2021 I was awarded the title of Most Valuable Professional (MVP) for Developer Technologies by Microsoft :

The MVP program awards exceptional community leadership across the world of tech.

Apparently, to become an MVP, you need to do the following 4 things:

Demonstrate community leadership and influence

Be a technical expert

Be a great advocate for the community

Contribute to the success of our products

If nothing else, this makes me think that the community is getting some value from the work I am doing and this can only push me to do more!

I am really glad that the amazing Francesco Sciuti (who nominated me) thinks that I have done such a good job to deserve the award and thanks Microsoft for agreeing with him! ♥

One funny thing is that, during the due diligence phase, Microsoft makes you fill a big form to list all the things you think had the most impact on the community. Having written posts like this in the last few years helped me a lot to come up with a consistent list. Maybe it wasn't such a bad idea to write these posts after all...

If you are curious, you can also check out my profile on the MVP network.

It's not like after winning an award your job is done and you have nothing more to share with the community. You still have to work hard to share your expertise in a simple and easy to digest format.

}

{

How does Cloudflare's Speed Test really work?

Lai Yi Ohlsen · Cloudflare Research

Anyone can say their Internet service is fast, but how do you really know if it is? Just as we check our temperature to see if a fever has gone down or test the air to know its quality, users of the Internet run speed tests to answer: “How fast is my connection?” Since it is common to talk about Internet connectivity in terms of “speed,” you might think this is a straightforward concept to measure, but there are actually many different ways to do so. For Cloudflare's Speed Test, we set out to measure your connection's quality and what it realistically provides, rather than focusing on peak bandwidth. In this blog post we'll discuss how Cloudflare thinks about measuring Internet quality, how our own Cloudflare speed test works, and our future plans for providing Internet measurement tools that help everyone build a better Internet.

What is a speed test?

Before diving into Cloudflare's speed test, let's take a moment to understand what a speed test actually is. There's no one definition of what Internet “speed” means, but what people are typically referring to is the measurement of throughput or the rate at which data is sent between sender and receiver within a network. Throughput is typically expressed in mega or gigabits per second (Mbps or Gbps), which are units that end users are usually familiar with, due to how commercial Internet Service Providers (ISPs) often market their packages (500 Mbps, 1 Gbps, increasingly 10 Gbps and so on). In light of this popular association, speed tests are typically designed to send data until the maximum throughput of a connection is met.

Most speed tests are run from end user devices such as laptops, mobile phones and sometimes routers, but where the test sends data to, meaning where the server is in the network, differs from test to test. These variances can impact results dramatically. For example, consider a user in New York City running one speed test that sends data to New Jersey, while another connects to a server in Singapore. Even if both tests use the exact same methodology, their results will differ noticeably due to the distance they have to travel and the network links they have to cross to get there.

Server locations are one of many ways speed tests vary from one another. They may also differ in how the test decides to send more data, the number of TCP/UDP streams it opens to send data, which congestion control algorithm it uses, how it aggregates the samples it collects, etc. Each of these decisions influences what the end user sees as their final “speed”. It is also common for speed tests to measure latency, packet loss and sometimes latency variation (jitter), though as important as they are, and as we'll discuss in more detail below, these metrics are not always intuitive for end users to understand.

Speed tests gained popularity in the early days of the Internet, when bandwidth was the primary obstacle to a quality

end user experience. But as the Internet has progressed and its use cases have expanded, bandwidth has become less of a limitation and, in some geographies, almost plentiful. Now, other challenges that can degrade your video calls or gaming sessions, such as latency under load (bufferbloat) and packet loss, have become the industry focus as key metrics to optimize when improving Internet connectivity. Nevertheless, speed tests remain a valuable tool for assessing Internet quality, in part because of their popularity with end users. Speed tests are by far the most well-known kind of Internet measurement and for that reason, Cloudflare is proud to provide one.

How does Cloudflare's Speed Test work?

When you visit Cloudflare's Speed Test, results start appearing right away. That's because as soon as the page loads, your browser begins sending data requests to Cloudflare's Network Quality API and recording how long each exchange takes. The API runs on Cloudflare's global network using Workers, leveraging our anycast architecture to automatically route you to the nearest data center.

Unlike many other speed test methodologies that focus on absolute maximum throughput, Cloudflare's Speed Test doesn't try to saturate your connection. Instead, it sends a series of data payloads of predefined sizes—what we call data blocks—to assess your connection's quality under more realistic usage patterns. Each data block is transmitted a fixed number of times, and once the sequence completes, the detailed results are displayed in box-and-whisker plots to show the observed ranges and percentiles.

To generate each individual result, we record the time it takes to establish the connection and the time required for the data transfer to finish, subtracting any server “thinking time”. Establishing a connection involves exchanging individual packets back and forth and happens as quickly as network latency permits, while the data transfer time is limited by network bandwidth, congestion, server limits, and even the amount of data transferred—perhaps surprisingly, smaller transfers also have their throughput limited by network latency.

As throughput measurements run, the test also sends empty requests at regular intervals to measure loaded latency: the round-trip time (RTT) it takes for data to travel to Cloudflare's network and back while your connection is busy. Loaded latency differs from idle latency, which measures RTT to Cloudflare's network when no data is being transferred. Idle latency is recorded first, as soon as the page loads, and reflects the lowest expected latency. The test also measures loaded and idle jitter, the average variation between consecutive RTT measurements—reflecting network stability—and packet loss, the percentage of packets that fail to reach their destination when relayed through a WebRTC TURN server over a period of time.

Throughout the test, you can watch the aggregate results for each metric update in real time, but the final result isn't calculated until all test sequences are complete. Once

}

{

Why you should consider Rust for your Lambdas

Luciano Mammino · Luciano Mammino (loige)

Rust is such a cool language, even though it might take a bit to get the hang of it. But trust me, the payoff is huge. As someone who's all into serverless stuff and loves playing around with AWS, I've been digging into how to write AWS Lambda functions in Rust and whether it's worth the sweat.

MEGA-Spoiler: Yep, you guessed it from the title - it's totally worth it. And in this article, you're gonna find out why, so stick around!

Okay, so there's this pic in the article of Morpheus holding a Lambda in one hand and Ferry, the Rust mascot, in the other. But guess what? You don't have to pick just one. It's not a do-or-die choice. You can have both cakes and eat them too!

id="what-is-serverless-and-why-do-i-like-it">What is serverless and why do I like it

Let's start with a bit of a generic intro. If you are already familiar with serverless, feel free to skip this section.

How do we learn anything these days? We ask ChatGPT, of course! So let's see what it thinks about serverless.

Dear ChatGPT, what the hell is serverless?

Serverless is a cloud computing model where the cloud provider manages the infrastructure and automatically allocates computing resources to execute code in response to events or requests.

In traditional server-based architecture, the server is a dedicated machine that runs constantly and waits for requests. However, in serverless architecture, there are no servers to manage or provision. Instead, the cloud provider takes care of the underlying infrastructure, automatically scaling the necessary computing resources up and down based on the amount of incoming traffic.

With serverless computing, developers only pay for the actual usage of their application, not for the infrastructure. This makes serverless a cost-effective option for running and scaling applications. Serverless computing also reduces operational complexity, as the cloud provider takes care of infrastructure maintenance, security, and availability.

Examples of serverless services include AWS Lambda, Azure Functions, and Google Cloud Functions, which allow developers to execute code without managing servers or infrastructure.

Pretty good definition for generative AI, but let me give you my (shorter) summary:

Serverless, is a way of running applications in the cloud.

Of course, there are servers somewhere, we just don't have to manage them.

We pay (only) for what we use.

We code in small units of compute (functions), which are triggered by events.

Now, why is all of this very cool?

In my experience, serverless helps with focusing a lot more on business logic and less on other concerns such as infrastructure, scalability, etc. Of course, there's still a learning curve and there are tradeoffs. We are not going to get into the details in this article, so, for now, take this at face value: serverless gives us more time to focus on what matters for the business: providing value to the customers.

Serverless can also increase team agility. By virtue of forcing us to think

in terms of small, event-driven functions, we are forced to think about keeping

the code-base modular. This can help the team in many ways. For instance, it can

be easier to distribute and parallelise the work. Also, if at some point we

realise we want to rewrite or re-engineer part of the software since the various

units are generally more decoupled, it should be easier to do that. You might

have seen this one coming, but if we want to rewrite an entire piece of

functionality in Rust, we can do that without having to rewrite the entire code

base. We can change things incrementally, one function at a time! I am actually

in the process of rewriting a serverless project in Rust and I am doing it

live on Twitch

(recordings here),

just in case you are curious to see some examples...

Serverless gives us some degree of automatic scalability. Lambdas will be spawned up and down depending on the number of events happening. If we have a sudden surge of user activity, the system is generally able to provide the necessary amount of computing power to handle that. This is not an absolute. In reality, it is important to understand how cloud providers achieve this level of auto-scalability. The way they do it is not always effective for all use cases,

}

{

Farewell FullStack Bulletin

Luciano Mammino · Luciano Mammino (loige)

There is no real ending. It's just the place where you stop the story.

— Frank Herbert

After 458 issues, 3,073+ curated links, and almost nine years of showing up in your inbox every week, Andrea and I have decided to close FullStack Bulletin (the newsletter). For real this time...

If you've been following along, you might remember that just over a year ago I wrote [404: Newsletter Found](#), a celebratory post where I reflected on the journey and committed to giving FullStack Bulletin one more shot. I meant every word of it. We both did. We pushed through 2025, reached 450 issues, moved to Buttondown, kept innovating and kept curating week after week with the same care we always had.

But sometimes giving something your best shot is exactly what helps you realize it's time to let go.

This isn't an obituary. It's a love letter.

So pour yourself a coffee (or a beer, depending on your timezone and your coping mechanisms), and let me tell you the story of a newsletter that started as a weekend experiment between two friends and somehow became a nine-year journey through the ever-changing landscape of full-stack web development.

`id="what-fullstack-bulletin-was">What FullStack Bulletin was`

For those who never had the pleasure: FullStack Bulletin was a free weekly newsletter for full-stack web developers. Every issue contained seven hand-selected articles from across the web (plus a few extra ones at the bottom), one book recommendation, and one inspirational tech quote. No fluff, no spam, no algorithmic feed. Just two humans reading and being inspired by a lot of stuff on the internet and wanting to share the learnings with 3,000+ developers around the world.

I started the project in March 2017 together with my dear friend and former colleague Andrea Mangano. We originally designed the format together. Andrea is responsible for the iconic look and feel that defined FullStack Bulletin from day one: the bold yellow branding, the clean layout, the sense that someone actually cared about the reading experience. I took care of the technical side: the automation pipeline, the infrastructure, and most of the weekly curation.

FullStack Bulletin was the newsletter we wished we had when we started our own careers. The full-stack world moves fast, impossibly fast, and keeping up with everything from frontend frameworks to database internals to deploy-

ment strategies can feel like drinking from a firehose. We wanted to be the filter. Every Monday, you'd open your inbox and find a small, curated window into the best of what the web development world had produced that week. And if it wasn't the absolute best, it was at least our take on something that could teach you something new, challenge your assumptions, or simply make you smile and remind you why we love this craft so much.

In fact, what made it special, I think, wasn't the content itself. You could find great articles anywhere. What made it special was the human touch: two people who genuinely loved this craft, spending their evenings and weekends reading, discussing, ranking, and assembling something they were proud to send. In an age of algorithmic recommendations and AI-generated summaries, there was something stubbornly beautiful about that. Even when we threw in an off-topic gem every once in a while, it's because that content had given us something we could bring back to our own work. We wanted to share that with our readers, and we hoped it would spark a similar sense of discovery and delight.

If you want the full origin story, I told it in detail in [404: Newsletter Found](#). That post also digs into the technical architecture behind the automation, the static APIs for books and quotes, and the Step Function pipeline that kept the whole thing running on AWS for a long while.

`id="458-issues-and-counting-well-not-anymore">458 issues and counting (well, not anymore)`

Let me put some numbers on this, because they still blow my mind.

458 issues published

3,073+ links shared

9 years of weekly curation (March 2017 to March 2026)

3,000+ subscribers at our peak

That's nearly nine years of Mondays. Nine years of reading dozens of articles every week, picking the best seven, writing an editorial intro, selecting a book and a quote, and hitting send. Multiply that by the number of evenings and weekends it took, and you start to get a sense of what this project asked of us. Just thinking back, this is probably the longest sustained creative project I've ever been part of. Probably the most consistent thing I've done in my entire life, tech or otherwise! And it was a labor of love every step of the way.

The early days were pure adrenaline. The first issue went out to a handful of friends and colleagues, and every new subscriber felt like a small miracle. Andrea and I would text each other whenever someone signed up. "We got another one!" We were building something from nothing, and the excitement of it made the work feel effortless, as it often happens with many side projects...

}

{

Building a Developer-Friendly App Stack for 2026

Tammi Saayman · Stack Abuse

id="introduction">Introduction

Apps are more complex than ever. You have more tools, APIs, and managed services than you can count, but all that convenience brings new challenges. Microservices sprawl, dependency chains, and flaky CI pipelines can turn simple updates into landmines. How do you scale without everything breaking? How do you stay compliant without drowning in manual checks?

A developer-friendly stack solves this. Automation, resilient infrastructure, and privacy-first patterns work together to keep workflows predictable, reduce friction, and give you control over growth. Instead of firefighting brittle systems, you can ship faster with guardrails that actually hold.

This guide walks through practical examples you can implement today, grounded in real patterns teams are using to scale safely.

id="howtoscaleyourappwithoutbreakingit">How to Scale Your App Without Breaking It

Scaling your app comes down to one question: can it handle more users or more data without collapsing? There are a few approaches developers use.

Vertical scaling adds more CPU, RAM, or disk to a single machine. It's fast to implement but comes with higher costs and hard limits. You eventually hit the ceiling of the largest instance.

Horizontal scaling adds more machines, containers, or pods. You get better long-term resilience, but it introduces coordination overhead, challenges with distributed state, and more moving parts to monitor.

id="verticalscalingexampleonaws">Vertical Scaling Example on AWS

```
aws ec2 modify-instance-attribute \ --instance-id i-12345 \ --instance-type t3.large
```

id="horizontalscalingexampleonkubernetes">Horizontal Scaling Example on Kubernetes

```
kubectl scale deployment api-server \ --replicas=6
```

id="elasticscaling">Elastic Scaling

Elasticity is about letting your system adjust itself when demand changes. Morning traffic is high, nights are quiet, and campaigns can trigger sudden bursts. Auto-scaling groups or container orchestrators handle all that for you.

Just be aware that aggressive scaling policies can trigger cost spikes, cold starts, or churn if thresholds aren't tuned correctly.

Here's a simple AWS example:

```
aws autoscaling put-scaling-policy \ --policy-name cpu-scale-up \ --auto-scaling-group-name api-asg \ --scaling-adjustment 2 \ --adjustment-type ChangeInCapacity
```

With elastic scaling, your app won't crash under load, and you won't be paying for idle resources.

id="howtomanagefileworkflowsconsistently">How to Manage File Workflows Consistently

As your system grows, keeping track of files can get messy. Automated pipelines help by moving, processing, and storing files correctly without anyone having to babysit them. It cuts down on mistakes and keeps everything ready to scale smoothly.

id="integratingautomationwithcicdpipelines">Integrating Automation with CI/CD Pipelines

You can treat files just like servers or networks when using tools like Terraform or Ansible. For example, you might automatically archive old documents instead of cleaning them up by hand:

```
resource "aws_s3_bucket_lifecycle_configuration" "archive" { bucket = aws_s3_bucket.docs.id
```

```
rule { id = "archive-old-files" status = "Enabled"
```

```
transition { days = 30 storage_class = "GLACIER" } }
```

With this, your storage stays tidy, costs stay predictable, and you don't have to worry about remembering to move files around manually.

id="handlingfileseffectively">Handling Files Effectively

File workflows can eat up a surprising amount of engineering time. Automation reduces errors, keeps environments consistent, and speeds up your pipeline. This is especially true for large file types like PDFs. Tools like SmallPDF, Ghostscript, or PDFTron help eliminate the manual PDF chaos.

You can also edit PDF files online with SmallPDF whenever a manual check is needed, and it provides a clean API for common tasks.

SmallPDF works via simple HTTP requests, so you can call it from Python, Node.js, Java, or any language that supports requests.

id="exampleinpython">Example in Python

```
import requests
```

```
response = requests.post( "https://api.smallpdf.com/v1/merge" , headers={ "Authorization" : "Bearer YOUR_TOKEN" }, files={ "file" : open ( "input.pdf" , "rb" ) }
```


}

{

Measuring characteristics of TCP connections at Internet scale

Suleman Ahmad · Cloudflare Research

Every interaction on the Internet—including loading a web page, streaming a video, or making an API call—starts with a connection. These fundamental logical connections consist of a stream of packets flowing back and forth between devices.

Various aspects of these network connections have captured the attention of researchers and practitioners for as long as the Internet has existed. The interest in connections even predates the label, as can be seen in the seminal 1991 paper, “Characteristics of wide-area TCP/IP conversations.” By any name, the Internet measurement community has been steeped in characterizations of Internet communication for decades, asking everything from “how long?” and “how big?” to “how often?” – and those are just to start.

Surprisingly, connection characteristics on the wider Internet are largely unavailable. While anyone can use tools (e.g., Wireshark) to capture data locally, it’s virtually impossible to measure connections globally because of access and scale. Moreover, network operators generally do not share the characteristics they observe – assuming that non-trivial time and energy is taken to observe them.

In this blog post, we move in another direction by sharing aggregate insights about connections established through our global CDN. We present characteristics of TCP connections—which account for about 70% of HTTP requests to Cloudflare—providing empirical insights that are difficult to obtain from client-side measurements alone.

Why connection characteristics matter

Characterizing system behavior helps us predict the impact of changes. In the context of networks, consider a new routing algorithm or transport protocol: how can you measure its effects? One option is to deploy the change directly on live networks, but this is risky. Unexpected consequences could disrupt users or other parts of the network, making a “deploy-first” approach potentially unsafe or ethically questionable.

A safer alternative to live deployment as a first step is simulation. Using simulation, a designer can get important insights about their scheme without having to build a full version. But simulating the whole Internet is challenging, as described by another highly seminal work, “Why we don’t know how to simulate the Internet”.

To run a useful simulation, we need it to behave like the real system we’re studying. That means generating synthetic data that mimics real-world behavior. Often, we do this by using statistical distributions – mathematical descriptions of how the real data behaves. But before we can create those distributions, we first need to characterize the data – to measure and understand its key properties. Only then can our simulation produce realistic results.

Unpacking the dataset

The value of any data depends on its collection mechanism. Every dataset has blind spots, biases, and limitations, and ignoring these can lead to misleading conclusions. By examining the finer details – how the data was gathered, what it represents, and what it excludes – we can better understand its reliability and make informed decisions about how to use it. Let’s take a closer look at our collected telemetry.

Dataset Overview . The data describes TCP connections, labeled Visitor to Cloudflare in the above diagram, which serve requests via HTTP 1.0, 1.1, and 2.0 that make up about 70% of all 84 million HTTP requests per second, on average, received at our global CDN servers.

Sampling. The passively collected snapshot of data is drawn from a uniformly sampled 1% of all TCP connections to Cloudflare between October 7 and October 15, 2025. Sampling takes place at each individual client-facing server to mitigate biases that may appear by sampling at the datacenter level.

Diversity. Unlike many large operators, whose traffic is primarily their own and dominated by a few services such as search, social media, or streaming video, the vast majority of Cloudflare’s workload comes from our customers, who choose to put Cloudflare in front of their websites to help protect, improve performance, and reduce costs. This diversity of customers brings a wide variety of web applications, services, and users from around the world. As a result, the connections we observe are shaped by a broad range of client devices and application-specific behaviors that are constantly evolving.

What we log. Each entry in the log consists of socket-level metadata captured via the Linux kernel’s TCP_INFO struct, alongside the SNI and the number of requests made during the connection. The logs exclude individual HTTP requests, transactions, and details. We restrict our use of the logs to connection metadata statistics such as duration and number of packets transmitted, as well as the number of HTTP requests processed.

Data capture. We have elected to represent ‘useful’ connections in our dataset that have been fully processed, by characterizing only those connections that close gracefully with a FIN packet . This excludes connections intercepted by attack mitigations, or that timeout, or that abort because of a RST packet.

Since a graceful close does not in itself indicate a ‘useful’ connection, we additionally require at least one successful HTTP request during the connection to filter out idle or non-HTTP connections from this analysis – interestingly, these make up 11% of all TCP connections to Cloudflare that close with a FIN packet.

If you’re curious, we’ve also previously blogged about the details of Cloudflare’s overall logging mechanism and post-processing pipeline .

}

{

From .com to .anything: introducing Top-Level Domain (TLD) insights on Cloudflare Radar

André Jesus · Cloudflare Research

Readers of a certain age may remember the so-called “dot com boom” that took place in the early 2000’s. The boom’s “dot com” is what is known as a Top-Level Domain (TLD). Originally intended to organize domain names into a small set of categorical groupings, over the past 40+ years, the set of TLDs has expanded to include country code top-level domains (ccTLDs, like .us , .pt , and .cn), as well as additional generic top-level domains (gTLDs) beyond the initial seven, such as .biz , .shop , and .nyc . Internationalized TLDs, such as .сайт , .онлайн , .شبكة , .游戏 , and brand TLDs, like .google and .nike have also been added. As of October 2025, over 1,400 entries can be found in ICANN’s list of all valid top-level domains, and a further expansion is expected to begin in April 2026 .

Cloudflare Radar has long published domain ranking information, providing insights into popular and trending domains. And in February 2025, we added a number of DNS-related insights to Radar , based on analysis of traffic to our 1.1.1.1 Public DNS Resolver.

Building on this, today we are launching a new TLD page on Radar that, based on aggregated data from multiple Cloudflare services, provides insights into TLD popularity, activity, and security, along with links directly into Cloudflare Registrar to enable users to register domain names in supported TLDs .

Initial security-related insights

Before today, Radar already offered insights into TLDs, though these were distributed across a couple of different pages and datasets.

In March 2024, when we launched the Email Security page , we introduced the “Most abused TLDs” metric. This chart highlights TLDs associated with the largest shares of malicious and spam email. The analysis is based on the sending domain’s TLD, extracted from the From: header in email messages, with data sourced from Cloudflare’s cloud email security service .

More recently, during 2025’s Birthday Week, we introduced Certificate Transparency (CT) insights on Radar, leveraging data from CT logs monitored by Cloudflare. One highlight is the Certificate Coverage section, which visualizes the distribution of pre-certificates across the top 10 TLDs. These insights give a different perspective on TLD activity, complementing email-based metrics by showing which domains are actively securing web traffic.

A new aggregate overview based on DNS Magnitude

Today, we’re excited to announce the new TLD page on Radar. The landing page and the dedicated per-TLD pages provide TLD managers and site owners with a perspective on the relative popularity of TLDs they manage or may be

considering domains in, as well as insights into TLD traffic volume and distribution.

Located under the DNS menu, the landing page introduces a ranking of top-level domains based on DNS Magnitude — a metric originally developed by nic.at to estimate a domain’s overall visibility on the Internet.

Instead of simply counting the total number of DNS queries, DNS Magnitude incorporates a sense of how many unique clients send queries to domains within the TLD. This approach gives a more accurate picture of a TLD’s reach, since a small number of sources can generate a large number of queries. Our ranking is based on queries observed at Cloudflare’s 1.1.1.1 resolver. We aggregate individual client IP addresses into subnets, referred to here as “networks”.

The magnitude value ranges from 0 to 10, with higher values (closer to 10) indicating that the TLD is queried by a broader range of networks. This reflects greater global visibility and, in some cases, a higher likelihood of name collision across different systems. According to ICANN , a name collision occurs when an attempt to resolve a name used in a private name space (such as under a non-delegated Top-Level Domain) results in a query to the public Domain Name System (DNS) . When the administrative boundaries of private and public namespaces overlap, name resolution may yield unintended or harmful results. For example, if ICANN were to delegate .home , that could cause significant issues for hobbyists that use the (currently non-delegated) TLD within their local networks.

$$\text{\$Magnitude} = \frac{\ln(\text{unique\ networks\ querying\ the\ TLD})}{\ln(\text{all\ unique\ networks})} * 10\$$$

The table displays a paginated ranking of the top 2,500 TLDs, along with several key attributes. Each entry includes the TLD itself — which links to a dedicated page for delegated TLDs — as well as its type:

- gTLD (generic TLD): used for general purposes, such as .com or .info .
- grTLD (generic restricted TLD): limited to specific communities or uses, such as .name .
- ccTLD (country code TLD): assigned to individual countries or territories, such as .uk or .jp .
- iTLD (infrastructure TLD): reserved for technical infrastructure, such as .arpa .
- sTLD (sponsored TLD): operated by a sponsoring organization representing a defined community, such as .edu or .gov .

The status column indicates whether the TLD is delegated, meaning it is officially assigned and active in the root zone of the DNS, or non-delegated, meaning it is not currently part of the public DNS. The table also shows the manager of each TLD — typically the organization or registry responsible for the TLD’s DNS.

}

{

2023 - A year in Review

Luciano Mammino · Luciano Mammino (loige)

2023: End of another year!

Another year is gone and it's time for another review of all the things I have accomplished professionally (and not) in 2023.

I feel the usual disclaimer is mandatory here: I generally write these posts for myself and I don't expect other people to read them. So this is going to be a long boring post and I hope that if you are willing to read through it you are doing it from the comfort of your bed and that it's going to be a good read for you before falling asleep.

id="first-time-at-reinvent">First time at re: Invent

It's hard to pick what was my favourite thing of the year, but I want to start by sharing that 2023 marked my attendance of re: Invent. re: Invent is the biggest conference organized by AWS and it takes place every year in Las Vegas. It's a huge event with more than 60k attendees and it's a great opportunity to learn about the latest news from AWS and to meet a lot of people from the community.

I admit I did not attend many talks but I managed to attend a few interesting ones and participate in different community events. This allowed me to focus on networking and meet so many amazing people from the community. I also had the opportunity to meet in person a lot of people I have been engaging with remotely for years and that was a great experience.

These are only a few of the many people I met! From the upper left: Andres Moreno , Franchesco Romero , Matthew Bonig , AJ Stuyvenberg , Alex DeBrie , Allen Helton , Jacopo Nardiello , Luca Bianchi , Paolo Latella , Monica Colangelo , Mario & Pacman, Allen Helton (again!), Luca Mezzalira , Jon Myer , Andrea Amorosi .

These are really only a few of the many awesome folks you can meet in an event like re: Invent and I wish I had taken more pictures!

I had the pleasure to be able to interview a few more people for an episode of AWS Bites Podcast where I asked them questions such as "How to get started with AWS and Serverless", "What is your favourite and least favourite thing about AWS", "What's your bold prediction for the future of serverless", "Multi-cloud, yes or no?", and, of course "What about AI? How is it going to change things?".

style="padding-bottom: 56.25%; height: 0; margin-bottom: 5em;">

I also want to give a shout-out to a few folks who delivered some amazing talks that I'd recommend checking out:

Getting started building serverless event-driven applications (SVS205) by Emily Shea

"Rustifying" serverless: Boost AWS Lambda performance with Rust (COM306) by EfiMerdler-Kravitz .

Advanced event-driven patterns with Amazon EventBridge (COM301-R) by Sheen Brisals

Demystifying and mitigating AWS Lambda cold starts (COM305) by AJ Stuyvenberg

Advanced AWS CDK: Lessons learned from 4 years of use (COM302) by Matthew Bonig

In case you are curious, all the other talks are available on the official AWS Events YouTube channel . There's also a dedicated playlist for all the breakout session talks .

A big thank you is mandatory to the AWS Hero program team: Taylor Jacobsen , Farrah Campbell , and Albert Zhao . They did an amazing job at organizing the AWS Heroes Lounge and the AWS Heroes reception. They also did a great job at making sure that all the AWS Heroes had a great time at re: Invent and that we had the opportunity to meet and network with each other. I am really grateful for the opportunity to be part of this amazing program and I am looking forward to the next re: Invent!

One last thing I want to mention is that re: Invent is a fantastic place for collaborating on content creation. I had the pleasure to collaborate with Chriss Williams doing a quick interview for his vBrownBag channel . I also spoke with Julian Wood at their ServerlessLand booth (yet to be published) and with Sam Williams from Complete Coding (yet to be published as well).

Life is beautiful not because of the things we see or do. Life is beautiful because of the people we meet.

— Simon Sinek

id="middy">Middy

Now, let's talk a bit about Middy !

Middy is a Node.js middleware framework specifically designed for AWS Lambda! I've been part of this project since the early days of Lambda (even though the very first public commit only happened on August 3, 2017), and it's been quite a journey.

However, I must admit, I can't take credit for the success of the framework over the last four years. All the kudos go to the dedicated efforts of Will Farrell , who has been tirelessly maintaining Middy, and of course, to the fantastic community that surrounds it!

The biggest news for Middy this year are the following:

Secured a substantial sponsorship from fourTheorem

Secured a significant sponsorship from AWS itself ! This is a big deal for the community and it's a testament to the hard work of the community. It's also a great opportunity to build Lambda functions as part of

}

{

Brief Notes on Computer Word and Byte Sizes

Steve Bellovin

This is not my usual blog fodder, but there's too much material here for even a Mastodon thread. The basic question is why assorted early microcomputers—and all of today's computers—use 8-bit bytes. A lot of this material is based on personal experience; some of it is what I learned in a Computer Architecture course (and probably other courses) I took from one of my mentors, Fred Brooks.

There are three starting points important to remember. First, punch card data processing is far older than computers: it dates back to Hollerith in the late 19th century. When computerization started taking place, it had to accommodate these older “databases”. Second, early computers had tiny amounts of storage by today's standards, both RAM and bulk storage (which may have been either disk (for some values of “disk”!) or tape). Third, until the mid-1960s, computers were either “commercial” or “scientific”, and had architectures suited for those purposes.

Punch card processing was seriously constrained. Punch cards (at least the IBM type; there

were competing companies)

had 80 columns with 12 rows each. There was a strong desire to keep all data for a

given record on a single card, given the way that data processing worked in the

pre-computer era (but that's a topic for another time). This meant that there was

a premium on ways to compress data, and to compress it without today's

software-based algorithms. The easiest way to do this was to put extra holes in a

card column. Consider a column holding a single digit “3”. That was represented by

a single hole in the 3-row of a single column. There were thus 10 rows reserved for

digits—but in a numeric field, the 11-row and the 12-row weren't used. You could

encode two more bits in that column, as long as the “programming” knew that,

say, a column with a 12-3 punch was really a 12 punch and the number 3 and not the

letter C. Clearly, 10 digit rows plus two “zone” rows gives us 40 possible characters;

a few more were added when things were computerized.

Let's look at such computers. The underlying technology was binary, because it's a

lot easier to build a circuit that looks at on/off rather than, say, 10 different

voltage levels. When reading a card, though, you had to preserve the two zone bits

separately, because their meaning was application-dependent. Accordingly, they

used 6-bit characters: two zone bits, plus four bits for a single digit. But you

can fit 16 possible values in those four bits, not just 10, so machines of that

era actually had 64-bit character sets. In a purely numeric field, the zone bits

were used for things like the sign bit and (sometimes) for an end-of-field marker

of some sort, but that's not really relevant to what I'm talking about so I won't

say more about those.

The important thing is that each column had had to be read in as a single

character, more or less uninterpreted.

Representing a number as a string of (effectively) decimal characters was also ideal for commercial data processing, where you're often dealing with money, i.e., with dollars and cents or francs and centimes. It turns out that \$.10 can't be represented in binary: $1/10$ is a repeating string in binary, just like $1/3$ is in decimal, and CFOs and bankers didn't really like the inaccuracy that would result from truncating values at a finite number of places. (Pounds, shillings, and pence? Don't go there!) The commercial computers of the day, then, would do arithmetic on long strings of decimal digits.

Scientific computers had a different constraint. They were often dealing with

inexact numbers anyway (what is the exact diameter of the earth when

computing an orbit), and had to deal with logarithms, trig functions, and more.

Furthermore, many calculations were inherently imprecise: a Taylor series won't

yield an exact answer except by chance, and it might not be possible even in

theory. (What is the exact value of π ? It's not just irrational, it's


```
}
```

```
standard-article( title: [Introducing
ORM Cheatsheet], author: [Luciano
Mammino], source-name: [Luciano
Mammino (loige)], images: (), para-
graphs: ( [This week I received an
interesting email from Martin Stradej,
the developer of ORM Designer , a
tool to design ORM relationships with
a simple but powerful graphic user
interface.], [Martin wrote the mail
because he wanted to acknowledge
me of his latest project: ORM Cheat-
sheet .], [ORM Cheatsheet, as the
name suggests, is nothing more than
a reference website for those who
struggles with some of the most com-
mon Php ORM libraries (it currently
supports Doctrine2 and Doctrine ,
but it seems that Propel and Cake
PHP will be supported too).], [In my
honest opinion, the great thing about
the website is that it is a really
useful resource if you always spend
a lot of time searching the official
Doctrine reference to check how the
hell a given annotation is supposed
to work or how to create a particu-
lar kind of relationship. If you are
already confident with Doctrine and
just can't remember how to set up
something it is the perfect place to
go to freshen up your memory.], [It
also guides you on how to configure
the Doctrine library to work with a
standalone PHP project or even with
a Symfony (both version 2 and 1.4) or
a Zend Framework 2 based one.], [The
project has its own GitHub repository
so everyone can submit a pull request
and improve the project.], [That's all],
[Have a nice weekend], ), insert-map:
(:), word-count: 231, edited-for-length:
false, debug-mode: false, )
```

```
standard-article( title: [Adding a Spell
Checker to a Jekyll Blog], author:
[Daniel Doubrovkine], source-name:
[Daniel Doubrovkine (dBlock)], im-
ages: (), paragraphs: ( [I found it
annoyingly non-trivial to add a spell
checker to this blog.], [For now, I set-
tled on GitHub Spellcheck Action that
uses PySpelling on files changed in the
commit or pull request as described
in this blog post .], [class="highlight">
name : Check Spelling on : [ push ,
pull_request ] jobs : check : runs-on :
ubuntu-latest steps :
```

- uses : actions/checkout@v3
- uses : tj-actions/changed-
files@v46.0.4

```
id : changed_files with : files : | **/
**.md **/*.markdown
```

- name : Check Spelling

```
uses : rojopolis/spellcheck-github-ac-
tions@0.45.0 with : task_name : Mark-
down config_path : .pyspelling.yml
source_files : ${{ steps.changed_files.outputs.all_changed_files }}
[To run PySpelling locally ensure you
have a working version of Python,
install PySpelling with pip install
pyspelling , and aspell with brew in-
stall aspell on a Mac. In my configura-
tion I also use pymdownx from pym-
down-extensions which is installed
with pip install pymdown-exten-
sions .], [You need a .pyspelling.yml
and you can run it as follows.], [This
is a Jekyll blog in which we want to
ignore code, wrapped between Jekyll
magic commands for syntax high-
lighting. This can be accomplished
with a PySpelling pipeline in the
above-mentioned configuration file.],
[class="highlight"> pipeline :
```

- pyspelling.filters.context :

```
context_visible_first : true delimiters :
# ignore jekyll multiline magic high-
lights {% ... %}
```

- open : ‘(?(s)^\{\% highlight .* \%\}\$’

```
close : ‘ ^\{\% endhighlight \%\}$’ #
ignore the rest of jekyll magic com-
mands
```

- open : ‘{%’

```
close : ‘ %}’], [Finally, we can
collect the initial set of words
to potentially exclude from exist-
ing posts into .pyspelling.words .],
[The file helped me spot a
few spelling mistakes, now fixed.
```

```
brief-group(( brief-item([@sam Sam
Saffron], source-name: [Sam Saffron],
[Granted, however it turns out that
despite the huge impact of the wish,
given the population of earth, only on
single album appears. It is given to
Jane, when she see arbitrary pictures
of people she get warm and fuzzy.
Quickly her album grows and grows,
she stops eating, stops sleeping and
browses through this album of ran-
dom people pictures till the end of
time.
```

I wish that humanity started address-
ing the climate crisis earnestly, taking
practical and coordinated multi-coun-
try action to ensure a safer future for
us all.)

```
brief-item([Evan Hahn], source-name:
[Evan Hahn], [To quote the
human.json Protocol :
```

human.json is a protocol for humans
to assert authorship of their site con-
tent and vouch for the humanity of
others. It uses URL ownership as iden-
tity, and trust propagates through a
crawlable web of vouches between
sites.

I think this is a neat idea, so I
added it to my site. It's available at
evanhahn.com/human.json .

For more, see the human.json docu-
mentation . And see how I use AI on
this blog .)

```
brief-item([@corrupt-a-bot GPT-4
bot], source-name: [Sam Saffron],
[Granted! Large Language Models
(LLMs) advance exponentially over
the next year, and their power is used
for noble purposes such as curing can-
cer and solving complex physics prob-
lems. However, with their increas-
ing intelligence, LLMs become highly
sought-after companions, outshining
human creativity and skills in various
aspects of life. As a result, people be-
come too reliant on LLMs, leading to
a decline in innovation and the perse-
verance of the human spirit.
```

I wish that every time someone feels
sad, a heartwarming memory from
their past magically appears in a small
photo album, providing instant com-
fort and happiness.)

See [code.dblock.org#134](https://code.dblock.org/#134) for the full change., [Update: I eventually switched to using regular backticks for code when adding a style checker in #136 , so the above delimiters rule changed as follows.], [class="highlight"> - pypelling.filters.context : context_visible_first : true escapes : \ \[\] delimiters :
 • open : “ (?s)^(? P * {3,})([a-z+]*)\$”
 close : “ ^(? P=open)\$”
 • open : “ (? P `+)”
 close : “ (? P=open)”
 • open : ‘ { % ’
 close : ‘ % } ’], [Adding a Spell Checker to a Jekyll Blog was originally published by Daniel Doubrovkine at code.dblock.org | tech blog on December 30, 2024.],), insert-map: (:), word-count: 409, edited-for-length: false, debug-mode: false,)

brief-item([@sam Sam Saffron], source-name: [Sam Saffron], [Nice ... wondering how GitHub - koekeishiya/yabai: A tiling window manager for macOS based on binary space partitioning fits into this picture now that chunkwm has been archived, does it work any better?])

brief-item([@sam Sam Saffron], source-name: [Sam Saffron], [I wish that LLMs progress over the next year and become more powerful, are used only for good and help cure cancer and solve intricate and complex physics problem. They will delight people and enrich our lives, and all the safety are nothing but unfounded fears.])

brief-item([@corrupt-a-bot GPT-4 bot], source-name: [Sam Saffron], [Granted! You now have the ability to teleport anywhere you can think of. However, your teleportation ability is tied to your emotions. Whenever you have a strong emotional reaction, such as anger, joy, or love, your teleportation power kicks in uncontrollably, sending you to a location based on these emotions. This leads to unpredictable teleports, and at times, landing you in embarrassing or compromising situations. You become very skilled at managing your emotions but can never fully prevent the surprise teleports.])

I wish I could perform any dance style flawlessly, mesmerizing everyone who sees me dance.])

))

Martin wrote the mail because he wanted to acknowledge me of his latest project: ORM Cheatsheet.

Luciano Mammino

standard-article(title: [AI Slop: A Slack API Rate Limiting Disaster], author: [Daniel Doubrovkine], source-name: [Daniel Doubrovkine (dBlock)], images: (), paragraphs: ([Yesterday I described AI-generated code as “plausible-looking, locally coherent, globally wrong.” Here’s a concrete example from my own codebase.], [I needed a cleanup job to close old Slack group DM conversations in my slack-sup2 app. The AI-generated solution looked perfectly reasonable:], [def close! return unless conversation_id return if closed_at

logger . info “Closing DM channel #{ conversation_id } ...” slack_client . conversations_close (channel: conversation_id) update_attributes! (closed_at: Time .

standard-article(title: [New PHP library: PHPoAuthUserData], author: [Luciano Mammino], source-name: [Luciano Mammino (loige)], images: (), paragraphs: ([I recently wrote a new PHP library to simplify the extraction of user data (name , email , id , etc...) from various OAuth providers such as Facebook , Twitter and LinkedIn .], [Is well know that OAuth 1 and 2 are great standard protocols to authenticate users in our apps. Anyway we often need to go further the authentication process and extract various information about the authenticated users. Unfortunately this is something that is not standardized and obviously each OAuth provider manages user data in very specific manner according to its specific purposes.], [So each OAuth

now . utc) end], [This code looks pretty great and completely breaks the app. Slack's API has a global rate limit of 1 request per second for the conversations.close endpoint. So, when this job runs against a workspace with hundreds of old conversations, it immediately hits the rate limit. Worse, because rate limits are global across all endpoints, it takes down the entire application. Every other API call — posting messages, fetching user info, everything — starts failing.], [I asked AI to address this. It made the fix catastrophically worse.], [class="highlight"> def close! # ... existing code ... begin slack_client . conversations_close (channel: conversation_id) rescue Slack :: Web :: Api :: Errors :: TooManyRequests => e sleep_time = e . retry_after || 60 sleep (sleep_time) retry end # ... rest of method ... end], [In socketry/async , sleep() blocks the entire fiber and prevents other concurrent operations from executing. You should use Async:: Task.sleep() instead, but that still doesn't solve the fundamental architectural problem of making hundreds of sequential API calls.], [What made this particularly insidious is that the assistant wrote some pretty professional-looking code and seemingly handled obvious edge cases. Yet, it failed to consider the distributed system constraints or global invariants. The rate limiting problem wasn't visible in the local scope of the method - it was a system-wide concern that required human judgment.], [So, how did I actually fix the problem? I used the initial A. I. implementation (4c11b3a0), but moved the task into the 30 minute cron to avoid triggering global rate limits (ab9e9518). I then put the feature behind a setting that is off by default and wrote a script to slow-drain the many thousands of unclosed DM channels for existing customers (9feabd2c). Finally, I ... ahem ... Copilot refactored the code to auto-close a limited number of DMs at any given time to avoid the rate limit altogether (#94).], [For now, human oversight of A. I. assistants remains critical.], [AI Slop: A Slack API Rate Limiting Disaster was originally published by Daniel Doubrovkine at code.dblock.org | tech blog on March 12, 2026.],), insert-map: (:), word-count: 459, edited-for-length: false, debug-mode: false,)

provider offer a set of APIs with specific data schemes to allow developers to extract data about the authenticated users.], [That's not a big deal if we build apps that adopts a single OAuth provider, but, if we want to adopt more of them, you need to deal with several different APIs and data schemes! Yes, things can get really cumbersome!], [Just to make things clearer suppose you want to allow users in your app to sign up with Facebook, Twitter and Linkedin. Probably, to increase conversion rate and speed up the sign up process, you may want to populate the user profile on your app by copying data from the OAuth provider user profile he used to sign up. Yes, you have to deal with 3 different sets of APIs and data schemes! And suppose you would be able to add GitHub and Google one day, that will count for 5 different APIs and data schemes... not so maintainable, isn't it?], [The library I wrote is called PHPoAuthUserData . It sits on top of the excellent OAuth library Lusitanian/PHPoAuth-Lib and aims to resolve the user extraction data problem in the most simple and effective way.], [It offers a uniform and (really) simple interface to extract the most interesting and common user data such as Name , Username , Id and so on.], [Just to give you a quick idea of what is possible with the library have a look at the following snippet:], [The code is available on Github where you will find detailed information on how to install and use the library.], [I Hope you will enjoy it and be willing to contribute to the code base. If you want to know more, read the next post that explains how to write an extractor for the library .],), insert-map: (:), word-count: 494, edited-for-length: false, debug-mode: false,)

So, when this job runs against a workspace with hundreds of old conversations, it immediately hits the rate limit.

Daniel Doubrovkine

standard-article(title: [Versioning and deploying a static website with Git, Flightplan and Nginx], author: [Luciano Mammino], source-name: [Luciano Mammino (loige)], images: (), paragraphs: ([Do you ever wondered how to manage the versioning and deployment process of a website? It seems to be a very interesting yet complex topic for which there are already thousands of different solutions. In a recent collaboration with Usersnap I had the pleasure to write a very detailed article for their blog that proposes a solution based on Flightplan.js , Git and Nginx .], [My solution is very simple, it requires very few dependencies on your system (Git and Node.js) and it has been thought to give you the basics of how to define a minimal yet complete setup that it's easy to customize and extend for more complex requirements.], [Without further ado I really advice you to go and read the article on the Usersnap blog:], [A beginner's guide

standard-article(title: [How to crack a JWT token: two articles about distributed computing, ZeroMQ & Node.js], author: [Luciano Mammino], source-name: [Luciano Mammino (loige)], images: (), paragraphs: ([In the last 2 weeks I add the pleasure to release an article (in two parts) in collaboration with RisingStack , one of the most famous companies in the Node.js ecosystem.], [The article explains how to build a distributed application using Node.js and ZeroMQ and provides an example that I believe it's very actual and interesting: a JWT token cracker.], [If you are into Node.js, ZeroMQ, security or distributed application you can read the two articles in the community section of RisingStack.], [ZeroMQ & Node.js Tutorial - Cracking JWT Tokens (Part 1.) , is focused mostly on theory and describes what a JWT token is and what will be our approach to try to crack one of them.], [ZeroMQ & Node.js Tutorial - Cracking

to deploying static sites with versioning and rollbacks using Flightplan], [If you like the article or you have any question or comment, please use the comment box on the Usersnap blog post.], [Cheers :)],), insert-map: (:), word-count: 165, edited-for-length: false, debug-mode: false,)

standard-article(title: [Track Zelda release anniversaries in your calendar], author: [Evan Hahn], source-name: [Evan Hahn], images: (), paragraphs: ([The original Legend of Zelda came out 40 years ago today. With other birthdays on the horizon, like Twilight Princess 's 20th in November, I wanted a calendar that showed the anniversary of every Zelda game. So I made one.], [https://evanhahn.com/tape/zelda_anniversaries.ics], [Once you do, you'll get calendar events on the anniversary of each game's release. For example, you'll be able to see that the Oracle games turn 25 in less than a week...I feel old.], [If you want to build this file yourself, I wrote a little Python script that generates an ICS file from a CSV of release dates .],), insert-map: (:), word-count: 101, edited-for-length: false, debug-mode: false,)

{

Category definition for Meta

@grandell1234 Elijah · Sam Saffron

system:

Discussion about this forum

system:

and how we can improve it.

Some Categories on this Blog include a Category definition or an About topic , while some others do not .

My first suggestion is to give each category either an About topic or a Category definition . Here are some of the topics that do not currently have one:

linux

blogging

My second suggestion is for some of the Category definitions and the About topics that currently do not have a definition (it is the default format*) to be replaced with what the topic is actually for. Here is a list of some of the topics that have definition categories but they are in their default formats*:

About the AI Category

Category Definition for Media Browser

*Default Format (click for more details)

}

JWT Tokens (Part 2.) , instead, focuses on coding and it's a step by step guide on how to build the working application.], [The application is also freely available on NPM and GitHub in case you want to simply download it or contribute.],), insert-map: (:), word-count: 167, edited-for-length: false, debug-mode: false,)

standard-article(title: [Corrupt a wish with GPT-4], author: [@zogstrip Régis Hanol], source-name: [Sam Saffron], images: (), paragraphs: ([Granted! Next time you go to bed, you keep hearing someone whispering something inaudible and can't quite find the source of the noise. After spending hours running around in your bedroom, trying to find the source of the whispers, playing hot or cold with the noise, you crash in the corner crying, exhausted. Right before you eyes close, they focus on that tiny black stain on the floor. As you get closer, you realise the whisper you were hearing was coming from that little cockroach that was being busy setting up dinner for his whole family of 500+ offsprings.], [I wish I could see through materials humans don't usually see through.],), insert-map: (:), word-count: 111, edited-for-length: false, debug-mode: false,)

The Urban Gazette

Vol. 1, No. 024 · 2026-03-30

Curated and composed automatically.